

05506026 Digital Image Processing (Java)

Lab 6

Objectives

1. Understand how to add salt, pepper, salt-and-pepper, and uniform noises.
2. Understand how to perform contraharmonic filter and alpha-trimmed mean filters.

Noises

Noises in digital images are unwanted information which come in the form of random colour information in images. Noises can be generated because of image sensor or data transmission. Noises can be problematic when we need to extract information in the image.

There are several types of noise. In this lab, we will try to add four types of noise.

Salt noises

Salt noises or white noises represent noises with extremely bright pixels. In other words, they are noises with only white colour.

The code to add salt noises is as below.

```

ImageManager.java
class ImageManager
{
    ...
    public void addSaltNoise(double percent)
    {
        if (img == null)    return;

        double noOfPX = height * width;

        int noiseAdded = (int)(percent * noOfPX);

        Random rnd = new Random();

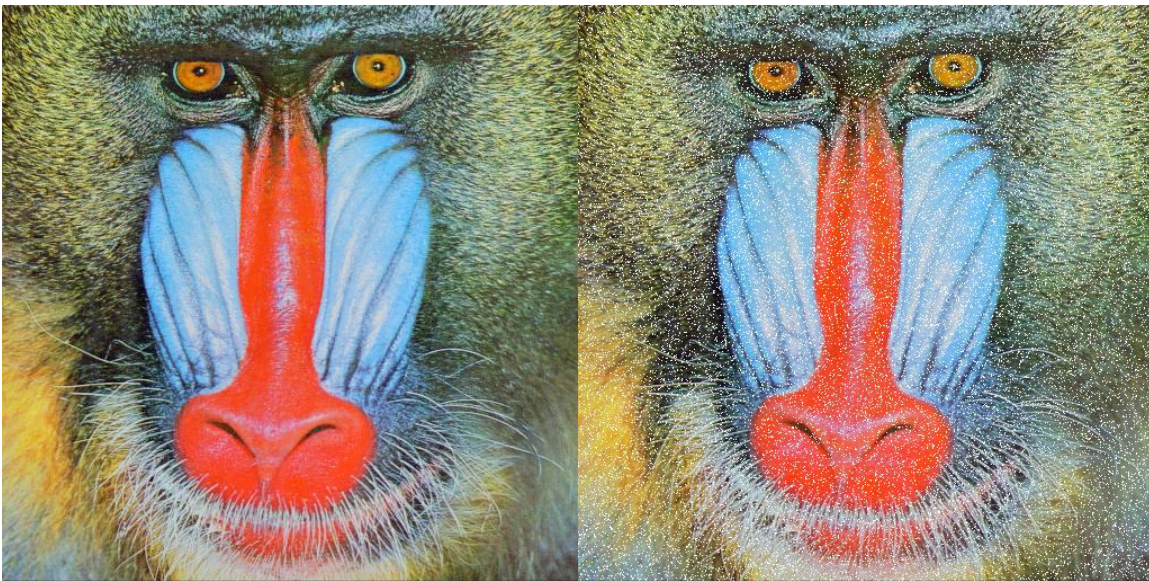
        int whiteColor = 255 << 16 | 255 << 8 | 255;

        for (int i = 1; i <= noiseAdded; i++)
        {
            int x = rnd.nextInt(width);
            int y = rnd.nextInt(height);

            img.setRGB(x, y, whiteColor);
        }
    }
    ...
}

```

Below is Mandril.bmp with 10% salt noises added.



Pepper noises

Pepper noises, in contrast with salt noise, represent noises with extremely dark pixels. In other words, they are noises with only black colour.

The code to add pepper noises is as below.

```

ImageManager.java
class ImageManager
{
    ...
    public void addPepperNoise(double percent)
    {
        if (img == null)    return;

        double noOfPX = height * width;

        int noiseAdded = (int)(percent * noOfPX);

        Random rnd = new Random();

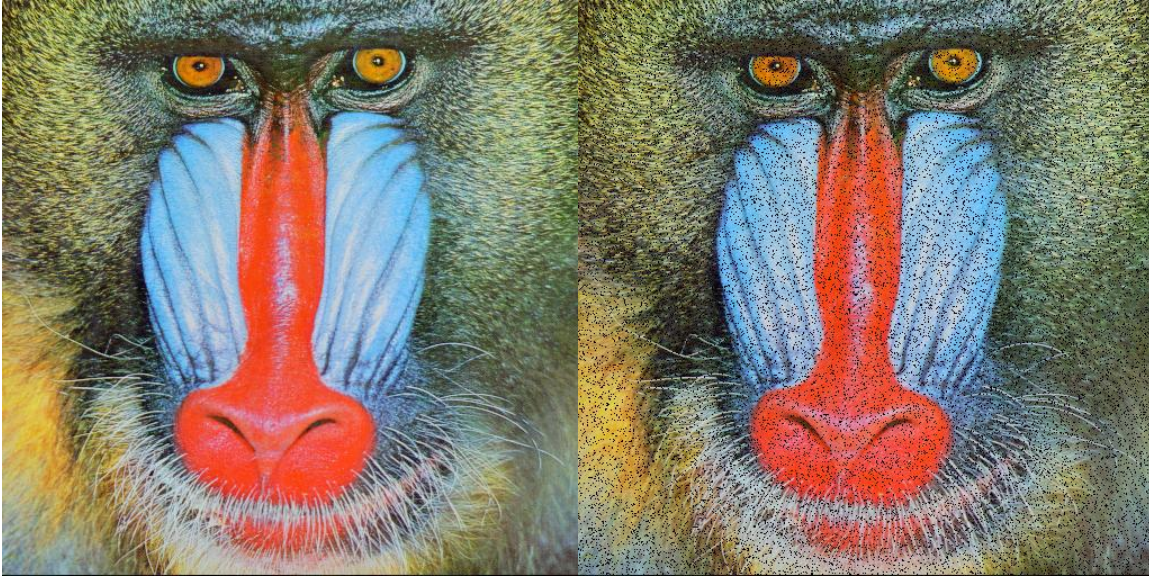
        int blackColor = 0;

        for (int i = 1; i <= noiseAdded; i++)
        {
            int x = rnd.nextInt(width);
            int y = rnd.nextInt(height);

            img.setRGB(x, y, blackColor);
        }
    }
    ...
}

```

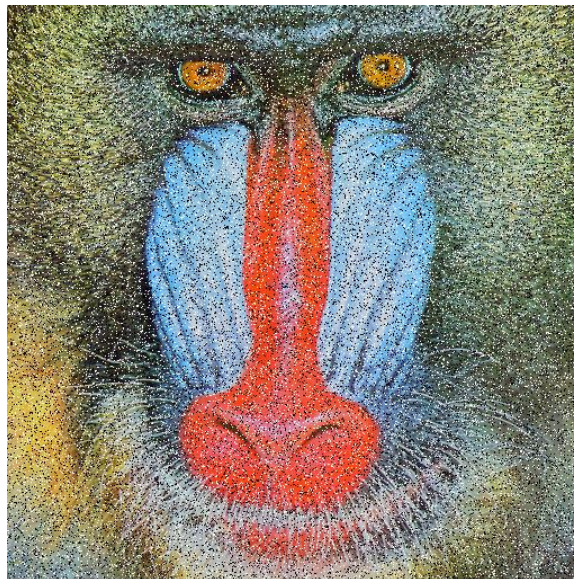
Below is Mandril.bmp with 10% pepper noises added.



Salt-and-pepper noises

Salt-and-pepper noises or impulse noises represent sharp and sudden noises in image signal. They are noises with white and black colours.

Below is Mandril.bmp with 10% salt noises and 10% pepper noises added.



Uniform noises

Uniform noises are noises which some variation of colour in pixels are altered. It happens with some pixel intensities are added with uniformly random number in range.

The code to add uniform noises is as below.

```

ImageManager.java
class ImageManager
{
    ...
    public void addUniformNoise(double percent, int distribution)
    {
        if (img == null)    return;

        double noOfPX = height * width;

        int noiseAdded = (int)(percent * noOfPX);

        Random rnd = new Random();

        for (int i = 1; i <= noiseAdded; i++)
        {
            int x = rnd.nextInt(width);
            int y = rnd.nextInt(height);

            int color = img.getRGB(x, y);
            int gray = color & 0xff;

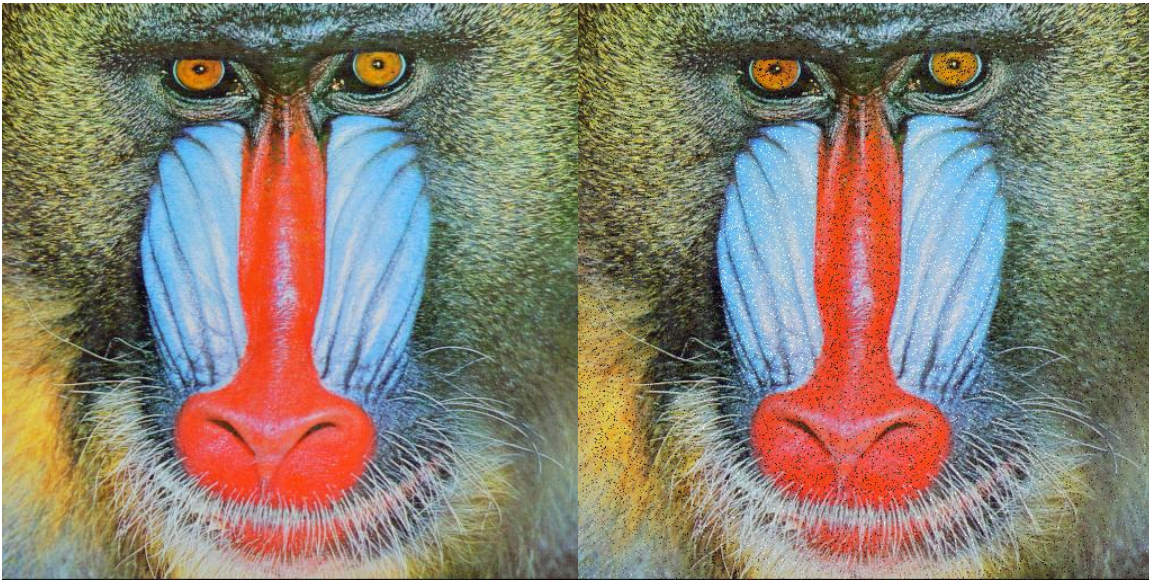
            gray += (rnd.nextInt(distribution * 2) - distribution);
            gray = gray > 255? 255: gray;
            gray = gray < 0? 0: gray;

            int newColor = gray << 16 | gray << 8 | gray;

            img.setRGB(x, y, newColor);
        }
    }
    ...
}

```


Below is Mandril.bmp with 10% uniform noises with variation of [-64, 64] added.



Contraharmonic filter

Contraharmonic filter is a spatial filtering to restore image from salt or pepper noise (one at the time, not both). The equation of contraharmonic filter is as below.

$$\hat{f}(x,y) = \frac{\sum_{(r,c) \in S_{xy}} g(r,c)^{q+1}}{\sum_{(r,c) \in S_{xy}} g(r,c)^q}$$

The code to perform contraharmonic filter is as below.

ImageManager.java

```

class ImageManager
{
    ...
    public void contraharmonicFilter(int size, double Q)
    {
        if (img == null)    return;

        if (size % 2 == 0)
        {
            System.out.println("Size Invalid: must be odd number!");
            return;
        }

        BufferedImage tempBuf = new BufferedImage(width, height, img.getType());

        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                double sumRedAbove = 0, sumGreenAbove = 0, sumBlueAbove = 0;
                double sumRedBelow = 0, sumGreenBelow = 0, sumBlueBelow = 0;

                for (int i = y - size/2; i <= y + size/2; i++)
                {
                    for (int j = x - size/2; j <= x + size/2; j++)
                    {
                        if (i >= 0 && i < height && j >= 0 && j < width)
                        {
                            int color = img.getRGB(j, i);
                            int r = (color >> 16) & 0xff;
                            int g = (color >> 8) & 0xff;
                            int b = color & 0xff;

                            sumRedAbove += Math.pow(r, Q + 1);
                            sumGreenAbove += Math.pow(g, Q + 1);
                            sumBlueAbove += Math.pow(b, Q + 1);

                            sumRedBelow += Math.pow(r, Q);
                            sumGreenBelow += Math.pow(g, Q);
                            sumBlueBelow += Math.pow(b, Q);
                        }
                    }
                }
            }
        }
    }
}

```

```

        sumRedAbove /= sumRedBelow;
        sumRedAbove = sumRedAbove > 255? 255: sumRedAbove;
        sumRedAbove = sumRedAbove < 0? 0: sumRedAbove;

        sumGreenAbove /= sumGreenBelow;
        sumGreenAbove = sumGreenAbove > 255? 255: sumGreenAbove;
        sumGreenAbove = sumGreenAbove < 0? 0: sumGreenAbove;

        sumBlueAbove /= sumBlueBelow;
        sumBlueAbove = sumBlueAbove > 255? 255: sumBlueAbove;
        sumBlueAbove = sumBlueAbove < 0? 0: sumBlueAbove;

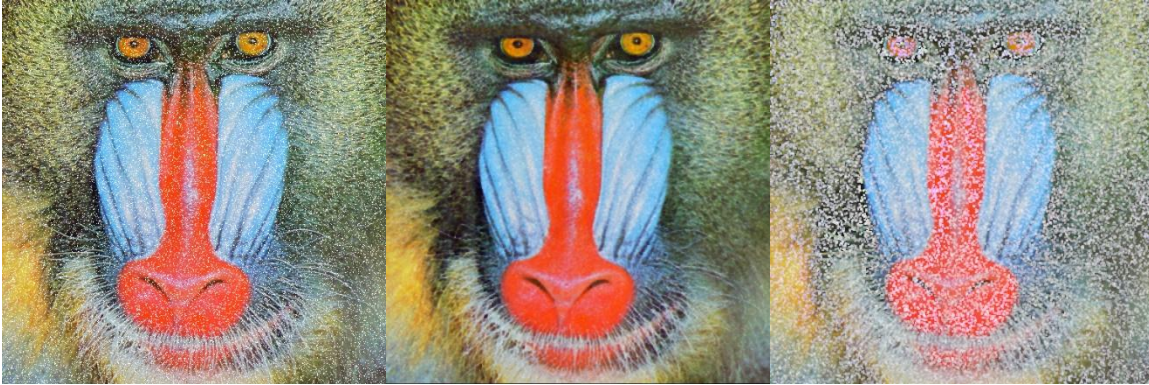
        int newColor = ((int)sumRedAbove << 16) | ((int)sumGreen-
Above << 8) | (int)sumBlueAbove;

        tempBuf.setRGB(x, y, newColor);
    }
}

for (int y = 0; y < height; y++)
{
    for (int x = 0; x < width; x++)
    {
        img.setRGB(x, y, tempBuf.getRGB(x, y));
    }
}
...
}

```

Contraharmonic filter can be used to restore images from salt or pepper noises, but we need to set order value (Q) properly. To reduce salt noises, the order value must be minus. Below is the example of Mandril.bmp with 10% white noises when we perform 3 x 3 contraharmonic filter with $Q = -1.5$ and 1.5 respectively.



Alpha-trimmed mean filter

Alpha-trimmed mean filter is a spatial filtering where d numbers of values in the kernel will be deleted ($d/2$ lowest and $d/2$ highest) and the rest of values are averaged after that. The equation of alpha-trimmed mean filter is as below.

$$\hat{f}(x, y) = \frac{1}{mn - d} \sum_{(r, c) \in S_{xy}} g_R(r, c)$$

The code to perform alpha-trimmed mean filter is as below.

ImageManager.java

```

class ImageManager
{
    ...
    public void alphaTrimmedFilter(int size, int d)
    {
        if (img == null)    return;

        if (size % 2 == 0)
        {
            System.out.println("Size Invalid: must be odd number!");
            return;
        }

        BufferedImage tempBuf = new BufferedImage(width, height, img.getType());

        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                int[] kernelRed = new int[size * size];
                int[] kernelGreen = new int[size * size];
                int[] kernelBlue = new int[size * size];

                for (int i = y - size/2; i <= y + size/2; i++)
                {
                    for (int j = x - size/2; j <= x + size/2; j++)
                    {
                        int r, g, b, k;
                        if (i >= 0 && i < height && j >= 0 && j < width)
                        {
                            int color = img.getRGB(j, i);
                            r = (color >> 16) & 0xff;
                            g = (color >> 8) & 0xff;
                            b = color & 0xff;

                            kernelRed[(i - (y - size/2)) * size + (j - (x - size/2))] = r;
                            kernelGreen[(i - (y - size/2)) * size + (j - (x - size/2))] = g;
                            kernelBlue[(i - (y - size/2)) * size + (j - (x - size/2))] = b;
                        }
                    }
                }

                for (int i = 0; i < size * size - 1; i++)
                {

```

```

    {
        int temp;

        if(kernelRed[j] > kernelRed[j+1])
        {
            temp = kernelRed[j];
            kernelRed[j] = kernelRed[j+1];
            kernelRed[j+1] = temp;
        }

        if(kernelGreen[j] > kernelGreen[j+1])
        {
            temp = kernelGreen[j];
            kernelGreen[j] = kernelGreen[j+1];
            kernelGreen[j+1] = temp;
        }

        if(kernelBlue[j] > kernelBlue[j+1])
        {
            temp = kernelBlue[j];
            kernelBlue[j] = kernelBlue[j+1];
            kernelBlue[j+1] = temp;
        }
    }
}

int remainingPixel = size * size - d;
int red = 0, green = 0, blue = 0;

for (int i = 0; i < remainingPixel; i++)
{
    red += kernelRed[(d / 2) + i];
    green += kernelGreen[(d / 2) + i];
    blue += kernelBlue[(d / 2) + i];
}

red /= remainingPixel;
red = red > 255? 255: red;
red = red < 0? 0: red;

green /= remainingPixel;
green = green > 255? 255: green;
green = green < 0? 0: green;

```

```

        blue /= remainingPixel;
        blue = blue > 255? 255: blue;
        blue = blue < 0? 0: blue;

        int newColor = (red << 16) | (green << 8) | blue;

        tempBuf.setRGB(x, y, newColor);
    }
}

for (int y = 0; y < height; y++)
{
    for (int x = 0; x < width; x++)
    {
        img.setRGB(x, y, tempBuf.getRGB(x, y));
    }
}
...
}

```

Alpha-trimmed mean filter can be used to restore images from several combined noises. Below is the example of Mandril.bmp with 10% salt noises, 10% pepper noises, and 10% uniform noises with variation of $[-64, 64]$ and we perform 5×5 alpha-trimmed mean filter with $d = 6$.



Quest 009

Implement the method of salt-and-pepper noises and use it to add 10% salt and 10% pepper noises.

The player who finishes this task will be awarded with **10 EXP**.

Deadline: 7 September 2025

Quest 010

Implement the method of contraharmonic filter and use it to perform 3×3 filter with $Q = -1.5$ and 1.5 on image that 10% pepper noises added.

The player who finishes this task will be awarded with **20 EXP** and **2 Moves**.

Deadline: 7 September 2025